

Centro Universitário UniFECAF
Análise e Desenvolvimento de Sistemas

Orçamento de Aluguel

Fluxograma, lógica e estrutura orientada a objetos

Nicolas Machado Castilhano

Algorithmic Thinking and Introduction to
Object-Oriented Programming

2026

1. Objetivo e regras de negócio

A imobiliária R.M. trabalha com locação de casas, apartamentos e estúdios, e precisa gerar orçamentos de aluguel para os clientes. Hoje esse cálculo é feito à mão, o que abre espaço para erro e faz o corretor gastar tempo com conta em vez de atendimento.

A aplicação desenvolvida automatiza esse cálculo. O usuário escolhe o tipo de imóvel e os opcionais, e o sistema devolve o valor mensal, o contrato já parcelado e um arquivo CSV com as doze parcelas do primeiro ano.

1.1 Tabela de preços

Item	Valor	Condição
Apartamento	R\$ 700,00	Valor base, com 1 quarto
Casa	R\$ 900,00	Valor base, com 1 quarto
Estúdio	R\$ 1.200,00	Valor base, sem divisão por quartos
Segundo quarto	R\$ 200,00	Somente apartamento
Segundo quarto	R\$ 250,00	Somente casa
Garagem	R\$ 300,00	Casa e apartamento
Estacionamento	R\$ 250,00	Somente estúdio, dá direito a 2 vagas
Vaga adicional	R\$ 60,00	Somente estúdio, a partir da terceira vaga
Desconto	5% do aluguel	Somente apartamento, para cliente sem crianças
Contrato	R\$ 2.000,00	Cobrança única, divisível em até 5 vezes

1.2 Duas decisões de interpretação

O enunciado deixa dois pontos em aberto, e a documentação registra a escolha feita em cada um deles.

Sobre o que incide o desconto de 5%. O texto fala em desconto "no valor do aluguel". A aplicação interpreta que o desconto incide sobre o aluguel já somado aos adicionais, e não apenas sobre o valor base. Assim, um apartamento de dois quartos com garagem sai de R\$ 1.200,00 para R\$ 1.140,00. Essa leitura é a mais favorável ao cliente e a mais coerente com a expressão usada no enunciado.

O que são as "12 parcelas do orçamento". A aplicação gera um arquivo com os doze meses do primeiro ano, mostrando em cada mês o aluguel, a parcela do contrato quando ela ainda estiver sendo cobrada, e o total do mês. Dessa forma o cliente enxerga que os primeiros meses são mais caros por causa do contrato, e que a partir do sexto mês o valor se estabiliza.

2. Pensamento algorítmico aplicado

Antes de escrever qualquer linha de código, o problema foi quebrado em partes menores. É esse processo que a disciplina chama de pensamento algorítmico, e ele aconteceu em quatro etapas.

2.1 Decomposição

O problema grande, "gerar um orçamento de aluguel", foi dividido em problemas pequenos e independentes:

1. Identificar quem é o cliente e se ele tem crianças.
2. Descobrir o valor base conforme o tipo de imóvel.
3. Somar os adicionais que o cliente escolheu.
4. Aplicar desconto, quando houver direito.
5. Calcular o contrato e dividir em parcelas.
6. Projetar os doze meses e exportar o arquivo.

Cada uma dessas etapas virou um método ou uma classe. Essa correspondência não é coincidência: quando a decomposição é bem feita, a estrutura do código quase se escreve sozinha.

2.2 Reconhecimento de padrões

Olhando as regras lado a lado, aparece uma repetição. Apartamento e casa seguem exatamente a mesma fórmula, mudando apenas os números: valor base, mais o segundo quarto, mais a garagem. O estúdio muda a fórmula, porque em vez de quartos e garagem ele tem vagas de estacionamento com uma regra escalonada.

Esse padrão levou à conclusão de que os três tipos compartilham um mesmo esqueleto e divergem só no meio do caminho. É exatamente o cenário em que herança resolve bem.

2.3 Abstração

Abstrair é separar o que é essencial do que é detalhe. O essencial: todo imóvel tem um valor base e sabe calcular o próprio aluguel. O detalhe: quanto custa o segundo quarto de uma casa. O essencial virou a classe abstrata `Imovel`. Os detalhes ficaram dentro de cada subclasse.

O ganho prático dessa separação aparece no arquivo `orcamento.py`. Ele calcula o orçamento sem conter uma única linha do tipo "se for apartamento". O orçamento só sabe que tem um imóvel na mão e pede o aluguel para ele.

2.4 Algoritmo

Com as partes definidas, a sequência de passos ficou assim, em pseudocódigo:

INÍCIO

```
LER nome, possui_crianças, tipo, opcionais, parcelas

SE nome estiver vazio OU campos numéricos forem inválidos ENTÃO
    EXIBIR mensagem de erro
    VOLTAR ao formulário
FIM SE

cliente <- NOVO Cliente(nome, possui_crianças)

ESCOLHA tipo
    CASO "apartamento": imovel <- NOVO Apartamento(quartos, garagem)
    CASO "casa"       : imovel <- NOVO Casa(quartos, garagem)
    CASO "estudio"    : imovel <- NOVO Estudio(vagas)
FIM ESCOLHA

adicionais <- imovel.calcular_adicionais()
subtotal   <- imovel.valor_base + adicionais

SE imovel é Apartamento E cliente NÃO possui crianças ENTÃO
    desconto <- subtotal * 0,05
SENÃO
    desconto <- 0
FIM SE

aluguel <- subtotal - desconto
contrato <- NOVO Contrato(parcelas)

PARA mes DE 1 ATÉ 12 FAÇA
    SE mes <= contrato.parcelas ENTÃO
        valor_contrato <- contrato.valor_total / contrato.parcelas
    SENÃO
        valor_contrato <- 0
    FIM SE
    tabela[mes] <- aluguel + valor_contrato
FIM PARA

EXIBIR aluguel, contrato e tabela

SE usuário pedir o arquivo ENTÃO
    GERAR arquivo CSV com as 12 linhas
FIM SE

FIM
```

Vale comparar o pseudocódigo com o código real. No pseudocódigo existe um `SE imovel é Apartamento`, porque em pseudocódigo isso é a forma natural de descrever a regra. No código em Python esse teste desaparece: a classe `Apartamento` sobrescreve o método `calcular_desconto`, e quem chama o método nem sabe que existe tratamento especial. Essa diferença é o ganho concreto da orientação a objetos sobre a programação puramente estruturada.

3. Fluxograma da aplicação

O fluxograma abaixo representa o caminho completo, do preenchimento do formulário até a geração do arquivo. Losangos são decisões, retângulos são processos e paralelogramos são saídas para o usuário.

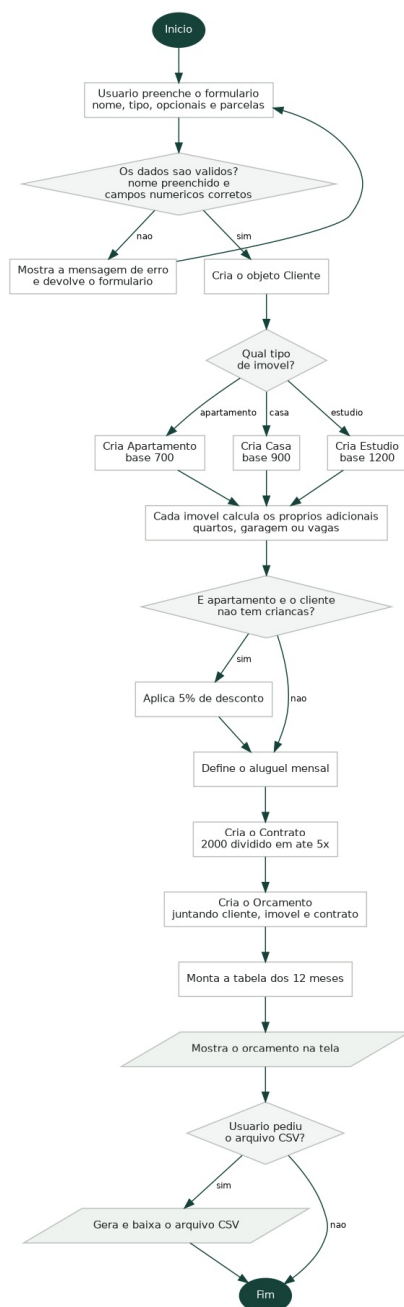


Figura 1. Fluxo principal da aplicação.

4. Estrutura orientada a objetos

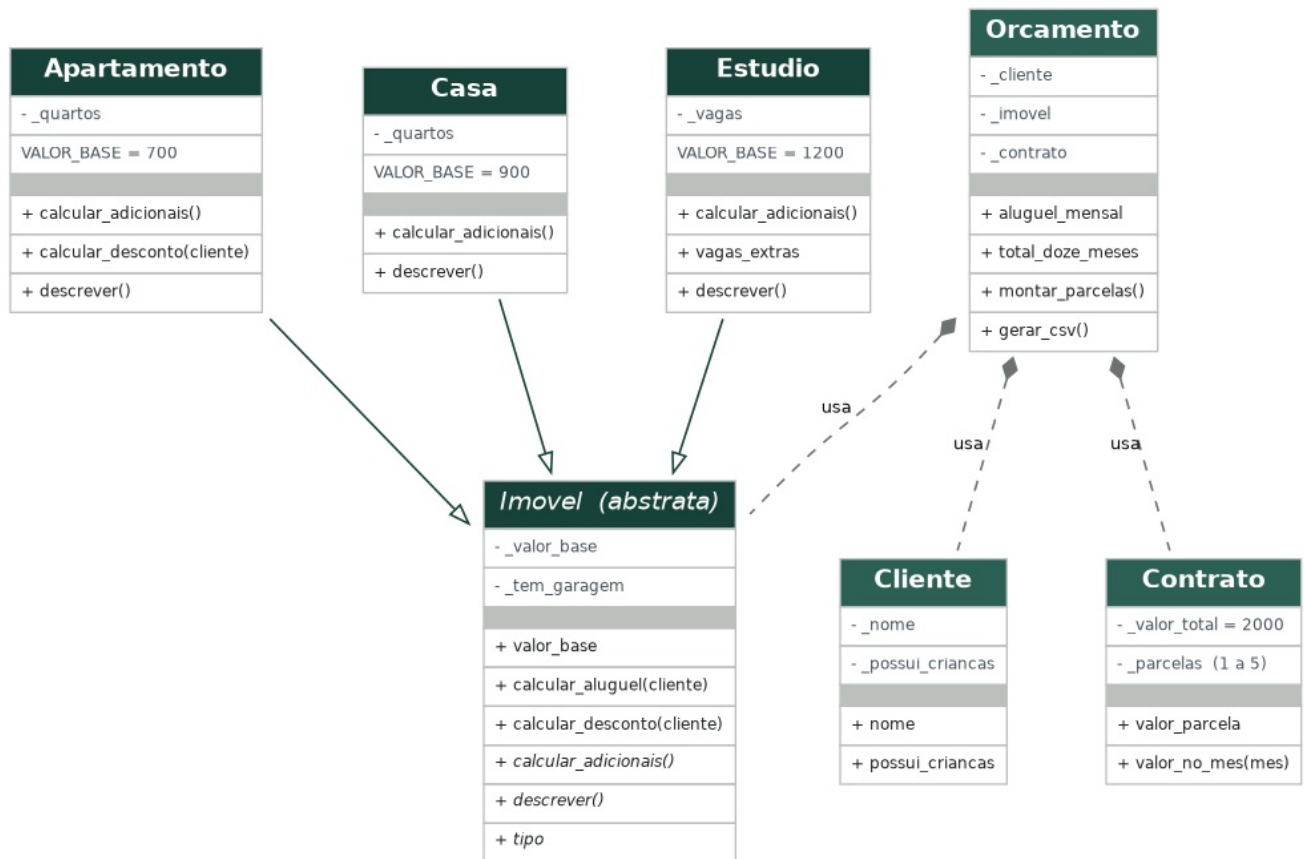


Figura 2. Diagrama de classes. Itálico indica método abstrato. A seta de ponta vazia indica herança e o losango indica composição.

4.1 Os quatro pilares no código

Abstração. A classe `Imovel` herda de `ABC` e declara três métodos com o decorador `@abstractmethod`. Isso torna a classe um contrato: ela define o que todo imóvel precisa saber fazer, sem dizer como. Tentar criar um `Imovel()` diretamente levanta erro, o que faz sentido, porque "imóvel" sozinho não existe no catálogo da R.M. O que existe é apartamento, casa ou estúdio.

Herança. As três classes concretas herdam de `Imovel` e reaproveitam tudo que já está pronto na classe mãe: o atributo de valor base, a propriedade de leitura e, principalmente, o método `calcular_aluguel`, que é escrito uma única vez e serve para os três tipos.

Polimorfismo. O método `calcular_adicionais` existe nas três subclasses com o mesmo nome e comportamentos diferentes. O trecho abaixo, que é o coração do sistema, funciona para qualquer tipo de imóvel sem saber qual deles recebeu:

```
def calcular_aluguel(self, cliente):
    subtotal = self.valor_base + self.calcular_adicionais()
    return round(subtotal - self.calcular_desconto(cliente), 2)
```

Essas três linhas atendem apartamento, casa e estúdio. Se amanhã a R.M. passar a alugar salas comerciais, basta criar a classe `Sala` herdando de `Imovel`. Este método não muda.

Encapsulamento. Os atributos começam com underline e são acessados por propriedades marcadas com `@property`. As validações moram dentro das próprias classes: quem tentar criar um apartamento de três quartos ou um contrato em seis vezes recebe um `ValueError` na hora. O resultado é que não existe objeto em estado inválido circulando pelo sistema.

4.2 Composição

A classe `Orçamento` não herda de ninguém. Ela é formada por três objetos que já sabem cuidar de si mesmos: um `Cliente`, um `Imovel` e um `Contrato`. Essa relação se chama composição, e a regra prática para escolher entre ela e a herança é simples: um apartamento é *um* imóvel, então usa herança; um orçamento *tem um* imóvel, então usa composição.

4.3 A fábrica de imóveis

O formulário devolve texto, mas o programa precisa de objetos. O arquivo `fabrica.py` é o único lugar de todo o projeto que decide qual classe instanciar, através de um dicionário que liga o texto do formulário à classe correspondente. Concentrar essa decisão em um só arquivo significa que adicionar um novo tipo de imóvel exige mexer em apenas dois lugares: criar a classe e registrá-la na fábrica.

5. Estrutura do projeto e validações

5.1 Organização dos arquivos

Arquivo	Responsabilidade
app.py	Interface. Recebe o formulário, monta os objetos e devolve a tela. Não contém nenhuma regra de negócio.
modelos/imovel.py	Classe abstrata e as três subclasses. Concentra os valores e as regras de cálculo.
modelos/cliente.py	Dados do cliente e a informação que dispara o desconto.
modelos/contrato.py	Valor do contrato e divisão em parcelas.
modelos/orcamento.py	Junta os três objetos, projeta os 12 meses e gera o CSV.
modelos/fabrica.py	Cria o objeto de imóvel correto e converte texto em número.
templates/ e static/	Telas em HTML e folha de estilo.

5.2 Validações implementadas

As validações acontecem em duas camadas. O navegador avisa cedo, mas quem decide é o servidor, porque a validação do navegador pode ser contornada por quem enviar a requisição por fora do formulário.

Situação	Resposta do sistema
Nome do cliente em branco	Recusa e pede o preenchimento
Texto onde se espera número	Recusa informando qual campo está errado
Número de vagas negativo	Recusa antes de criar o objeto
Apartamento ou casa com quantidade de quartos diferente de 1 ou 2	A própria classe levanta ValueError
Contrato dividido em mais de 5 vezes	A classe Contrato recusa
Tipo de imóvel não informado	A fábrica recusa

5.3 Geração do arquivo CSV

O arquivo é montado em memória com `io.StringIO` em vez de ser gravado direto no disco. Assim o mesmo método serve tanto para salvar um arquivo quanto para entregar o download pelo navegador. O separador escolhido foi o ponto e vírgula, e o decimal usa vírgula, que é o formato que o Excel em português abre corretamente sem precisar de configuração.

6. Considerações finais

O trabalho cumpre as funcionalidades pedidas: os três tipos de locação com seus valores, os adicionais de quarto e garagem, a regra escalonada de vagas do estúdio, o desconto condicional de 5%, o contrato parcelado em até cinco vezes e a exportação das doze parcelas em CSV.

O aprendizado principal, porém, não foi a conta em si, que é simples. Foi perceber onde cada regra deve morar. A primeira versão pensada para este projeto colocava todos os cálculos em uma função só, cheia de condicionais encadeadas para diferenciar apartamento, casa e estúdio. Funcionaria, mas cada nova regra da imobiliária significaria mais um `if` dentro da mesma função, até ninguém mais conseguir ler o que estava escrito.

Ao distribuir as regras entre classes, cada tipo de imóvel passou a responder apenas pelo que é seu. O resultado é que o arquivo que calcula o orçamento não precisa conhecer nenhum dos três tipos, e adicionar um quarto tipo não exige alterar nada do que já funciona. Esse é, na prática, o motivo de a orientação a objetos existir.